

Interrupts and Timers

TUE 1-3PM Weekly | Spring
2025

Attendance

Earn a Certificate of
Completion by attending
at least 70% of the
workshops



linktr.ee/CODE.CRUNCH



Interrupts

- They are a mechanism used to handle asynchronous events
- instead of checking constantly for a condition, the system “interrupts” the normal flow, handles an event and then resumes the previous task
- We interrupt and then continue the code

How interrupts work

- an event triggered by hardware or software
 - timer reaching a value
 - button press
 - data received on communication
 - associated with specific Interrupt Service Routine
- processor pauses current program execution
 - saves current state to a stack
- ISR runs to handle the event
 - should be short and is a high-priority function
- After ISR finishes, processor restores the saved state and continues the main program

Types of Interrupts

- Hardware interrupts
 - triggered by external events
 - buttons
 - signals from sensors or outside devices
- Software Interrupts
 - triggered by the software or instructions
 - Timers to initiate interrupts at specific times

Characteristics of Interrupts

The logo for 'CODE CRUNCH' is located in the top right corner. 'CODE' is written in a blue, stylized font with a yellow star above the 'O'. 'CRUNCH' is written in a red, stylized font with a yellow star above the 'H'. The logo is tilted upwards to the right.

- Asynchronous
 - should occur at any time, independent of main program flow
- Prioritized
 - some systems allow multiple interrupt sources, each with a priority level
- Atomicity
 - While an ISR runs, interrupts are typically disabled to prevent nested execution
- Short and fast
 - they block the main program and can delay other interrupts
- Avoid using global variables
- Do not overuse complex logic or long operations/ delay functions

Timers

- They measure time intervals, generate periodic events, create precise delays
- operate independently of main CPU
 - this allows you to generate PWM signals, create delays, or trigger interrupts without constant software intervention



How Timers Work

- Clock Source
 - Timers count pulses from a clock source, internal or externally to the system
- Counter Register
 - increments or decrements with each clock pulse, this is the current value of the timer
- Prescaler
 - divides clock frequency to control how quickly the timer increments
 - Ex. prescaler of 1024 reduces a 16MHz clock to 15.625 kHz
- Compare or Overflow
 - Timers generate events when they:
 - Match a specific value to the compare register
 - Overflow(counter exceeds max value)

Types of Timers

- 8 Bit timers
 - range from 0 to 255
 - faster but limited precision
 - Timer0, Timer2 on AVR based microcontroller
- 16 bit timers
 - ranges from 0 to 65,535
 - more precise but slower
 - Timer1 on AVR based microcontroller
- 32 bit Timer
 - Larger range for long timing intervals

Timer Modes

- Normal mode
 - timer increments till it overflows then it resets
 - useful for measuring time intervals
- CTC Mode(Clear Timer on Compare Match)
 - timer resets to 0 when it matches a compare value in the register
 - ideal for periodic events
- PWM mode
 - generate Pulse Width Modulation signals for controlling motors, LEDs
- Input Capture Mode
- Captures the timer value when an external event occurs



Example Setup

```
cli(); // Disable global interrupts TCCR1A = 0; // Normal mode
TCCR1B = (1 << WGM12) | (1 << CS12) | (1 << CS10); // CTC
mode, prescaler 1024 OCR1A = 15624; // Compare match value
for 1 second TIMSK1 = (1 << OCIE1A); // Enable compare match
interrupt sei(); // Enable global interrupts
```



Q&A Session



Attendance



Join us for the upcoming weekly sessions this Spring 2025!



Monday: C5 1PM / C3 2PM
Tuesdays: C7 1PM
Wednesdays: C5 1PM / C1 2PM
Thursdays: C6 1PM / C4 2PM / C8 3PM
Friday: C2 1PM & C9 2PM



RSVP LU.MA/CODECRUNCH



Your feedback helps us improve. Share your thoughts!

Go to our website: go.fiu.edu/codecrunch